

Résumé Complet - Développement Android

Du Chapitre 1 au Chapitre 9 - Révision Examen

AmineGR03

4 février 2026

Table des matières

1	Chapitre 1 : Introduction à Android	2
1.1	Qu'est-ce qu'Android ?	2
1.1.1	Caractéristiques principales	2
1.1.2	Architecture Android	2
1.2	Composants fondamentaux d'une application Android	2
1.2.1	Activity (Activité)	2
1.2.2	Service	3
1.2.3	BroadcastReceiver	3
1.2.4	ContentProvider	3
1.3	AndroidManifest.xml	3
2	Chapitre 2 : Cycle de Vie et Activities	4
2.1	Le Cycle de Vie d'une Activity	4
2.1.1	Méthodes du cycle de vie	4
2.1.2	Exemple complet d'une Activity	4
2.2	Configuration Changes	5
3	Chapitre 3 : Interface Utilisateur sous Android (Partie 1)	6
3.1	Introduction aux Vues et Contrôles	6
3.1.1	Concepts fondamentaux	6
3.1.2	Contrôles basiques	6
3.1.3	Contrôles de sélection	7
3.2	Positionnement et Layouts	8
3.2.1	Types de Layouts	8
3.2.2	Attributs des Layouts	9
3.2.3	Identificateurs (ID) Android	9
4	Chapitre 3 : Interface Utilisateur (Partie 2) - TRÈS IMPORTANT	9
4.1	Élasticité et Adaptabilité des UI	9
4.1.1	Unités de mesure	9
4.1.2	Bonnes pratiques UI adaptatives	10
4.2	ConstraintLayout - Approfondissement	10
4.2.1	Types de Contraintes	10
4.3	Listes et Adaptateurs	11
4.3.1	Définition	11
4.3.2	Principe du pattern Adapter	11
4.3.3	Types courants	11
4.3.4	Exemple avec ListView et ArrayAdapter	11

4.4	RecyclerView - Approfondissement	12
4.4.1	Exemple RecyclerView complet	12
4.5	Navigation entre Activités	14
4.5.1	Intent	14
4.5.2	Navigation simple	14
4.5.3	Transfert de données	15
4.6	Gestion des Événements	15
4.6.1	OnClickListener	15
4.6.2	OnFocusChangeListener	16
4.6.3	TextWatcher	16
4.7	Les Menus	17
4.7.1	Menus d'options	17
4.7.2	Menus contextuels	17
4.8	Dialogue avec l'Utilisateur	18
4.8.1	Toast	18
4.8.2	AlertDialog	18
4.8.3	Snackbar	19
5	Chapitre 4 : Persistance des Données	19
5.1	Introduction	19
5.2	Accès aux Fichiers	20
5.2.1	Types de stockage	20
5.2.2	Écriture dans un fichier interne	20
5.2.3	Lecture d'un fichier interne	20
5.2.4	Bonnes pratiques pour la gestion des fichiers	21
5.3	SharedPreferences	21
5.3.1	Obtenir l'Instance	21
5.3.2	Lire des Données	21
5.3.3	Écrire des Données	21
5.3.4	Supprimer des Données	22
5.3.5	apply() vs commit()	22
5.4	Bases de Données SQLite	22
5.4.1	Caractéristiques	22
5.4.2	SQLiteOpenHelper	22
5.4.3	Création d'une classe DatabaseHelper	22
5.4.4	Opérations CRUD	23
5.4.5	Utilisation dans une Activity	24
6	Chapitre 5 : Communication et Threads	25
6.1	Thread Principal (UI Thread)	25
6.2	Multi-threading	25
6.3	Handler	25
6.4	AsyncTask (Déprécié)	26
6.5	ExecutorService (Recommandé)	26
7	Chapitre 6 : Intents et Communication entre Composants	26
7.1	Introduction aux Intents	26
7.2	Les Intents Explicites	26
7.3	Les Intents Implicites	26
7.4	Composantes d'un Intent	27
7.5	Exemples d'Actions Prédéfinies	27
7.6	Exemples Pratiques	27

7.6.1	Ouvrir une Page Web	27
7.6.2	Faire un Appel Téléphonique	27
7.6.3	Partager du Texte	27
7.7	Passage de Données entre Activités	27
7.7.1	Envoi d'Extras	27
7.7.2	Récupération	28
7.7.3	Transmission d'Objets Complexes (Parcelable)	28
7.8	Activity Result API	28
7.9	Broadcast Receivers	29
7.9.1	Types	29
7.9.2	Exemple	29
7.10	PendingIntent	29
8	Chapitre 7 : API Android	30
8.1	Content Providers et Accès aux Contacts	30
8.1.1	Permissions nécessaires	30
8.1.2	Exemple : Lire les contacts	30
8.2	Fonctions de Téléphonie	30
8.2.1	Appels téléphoniques	30
8.2.2	Envoi de SMS	30
8.3	Géolocalisation avec Android	31
8.3.1	Sources de localisation	31
8.3.2	APIs de localisation	31
8.3.3	Permissions de localisation	31
8.4	Caméra	31
8.5	Multimédia	31
8.5.1	Sonneries et Vibrations	31
9	Chapitre 8 : Communication Réseau sous Android	32
9.1	Établissement de Connexions	32
9.2	Protocoles de Communication	32
9.2.1	HTTP/HTTPS	32
9.3	Web Services	32
9.3.1	REST et JSON	32
9.3.2	Retrofit	32
9.4	Communication Réseau - Suite	33
9.4.1	OkHttp	33
9.4.2	Volley (Alternative)	33
9.4.3	Gestion des erreurs réseau	34
9.4.4	Bonnes pratiques	34
10	Chapitre 9 : Services et Notifications	34
10.1	Introduction aux Services	34
10.1.1	Types de Services	34
10.2	Started Service	35
10.2.1	Création d'un Service	35
10.2.2	Flags de retour	35
10.3	Bound Service	35
10.3.1	Exemple avec Binder	35
10.4	Foreground Service	37
10.4.1	Création	37
10.5	Notifications	37

10.5.1	Création d'un canal de notification (Android 8.0+)	37
10.5.2	Création d'une notification simple	38
10.5.3	Notification avec action	38
10.5.4	Types de notifications	38
10.5.5	Notification BigText	38
10.5.6	Notification avec progression	39
10.6	JobScheduler et WorkManager	39
10.6.1	WorkManager (Recommandé)	39
10.7	Bonnes pratiques	39
10.8	Permissions pour les Services	40
11	Conclusion	41

Chapitre 1 : Introduction à Android

Qu'est-ce qu'Android ?

Android est un système d'exploitation open-source basé sur le noyau Linux, principalement conçu pour les appareils mobiles tactiles (smartphones, tablettes). Il a été développé par Google et l'Open Handset Alliance.

Caractéristiques principales

- **Open Source** : Le code source est disponible publiquement
- **Multi-plateforme** : Fonctionne sur smartphones, tablettes, TV, montres, etc.
- **Basé sur Linux** : Utilise le noyau Linux pour la gestion système
- **Java/Kotlin** : Développement principalement en Java ou Kotlin
- **Google Play Store** : Plateforme de distribution d'applications

Architecture Android

Android suit une architecture en couches :

1. **Applications** : Applications installées par l'utilisateur
2. **Framework Android** : API Java pour développer des applications
3. **Bibliothèques natives** : Bibliothèques C/C++ (SQLite, WebKit, etc.)
4. **Android Runtime (ART)** : Machine virtuelle pour exécuter les applications
5. **Noyau Linux** : Gestion des pilotes, mémoire, processus

Composants fondamentaux d'une application Android

Activity (Activité)

Une **Activity** représente un écran unique avec une interface utilisateur. C'est le composant principal pour l'interaction utilisateur.

Caractéristiques :

- Chaque écran de l'application est une Activity
- Une application peut avoir plusieurs Activities
- Une Activity a un cycle de vie bien défini
- L'Activity principale est déclarée dans le `AndroidManifest.xml`

Exemple de déclaration dans `AndroidManifest.xml` :

```
1 <activity
2     android:name=".MainActivity"
3     android:label="@string/app_name">
4     <intent-filter>
5         <action android:name="android.intent.action.MAIN" />
6         <category android:name="android.intent.category.LAUNCHER" />
7     </intent-filter>
8 </activity>
```

[**TIP**] : L'Activity avec `LAUNCHER` est celle qui s'affiche au démarrage de l'application.

Service

Un **Service** est un composant qui exécute des opérations en arrière-plan sans interface utilisateur.

Types de Services :

- **Started Service** : Démarré par `startService()`, continue même si l'Activity est détruite
- **Bound Service** : Lié à un composant via `bindService()`, détruit quand tous les clients se déconnectent

BroadcastReceiver

Un **BroadcastReceiver** permet à l'application de recevoir des messages système (batterie faible, SMS reçu, etc.).

ContentProvider

Un **ContentProvider** permet de partager des données entre applications de manière sécurisée.

AndroidManifest.xml

Le fichier `AndroidManifest.xml` est le fichier de configuration principal de l'application Android.

Éléments essentiels :

- Déclaration des composants (Activities, Services, etc.)
- Permissions requises
- Configuration de l'application (nom, icône, version)
- API minimum et cible

Exemple minimal :

```
1 <?xml version="1.0" encoding="utf-8"?>
2 <manifest xmlns:android="http://schemas.android.com/apk/res/android"
3     package="com.example.monapp">
4
5     <uses-permission android:name="android.permission.INTERNET" />
6
7     <application
8         android:allowBackup="true"
9         android:icon="@mipmap/ic_launcher"
10        android:label="@string/app_name"
11        android:theme="@style/AppTheme">
12
13         <activity android:name=".MainActivity">
14             <intent-filter>
15                 <action android:name="android.intent.action.MAIN" />
16                 <category android:name="android.intent.category.LAUNCHER" />
17             </intent-filter>
18         </activity>
19     </application>
20 </manifest>
```

[ATTENTION] Attention : Tous les composants doivent être déclarés dans le manifest, sinon ils ne seront pas accessibles.

Chapitre 2 : Cycle de Vie et Activities

Le Cycle de Vie d'une Activity

Le cycle de vie d'une Activity est crucial à comprendre. Android appelle automatiquement des méthodes à différents moments.

Méthodes du cycle de vie

1. **onCreate()** : Appelée lors de la création de l'Activity
 - Initialisation de l'interface utilisateur
 - Récupération des données sauvegardées
 - Configuration initiale
2. **onStart()** : L'Activity devient visible mais pas encore interactive
3. **onResume()** : L'Activity est au premier plan et interactive
 - L'utilisateur peut interagir
 - Reprendre les animations, caméra, etc.
4. **onPause()** : L'Activity perd le focus
 - Une autre Activity apparaît partiellement
 - Sauvegarder les données temporaires
5. **onStop()** : L'Activity n'est plus visible
 - Une autre Activity occupe tout l'écran
 - Libérer les ressources coûteuses
6. **onRestart()** : L'Activity redémarre après onStop()
7. **onDestroy()** : L'Activity est détruite
 - Nettoyage final
 - Libération de toutes les ressources

Exemple complet d'une Activity

```
1 public class MainActivity extends AppCompatActivity {
2
3     private static final String TAG = "MainActivity";
4     private TextView textView;
5
6     @Override
7     protected void onCreate(Bundle savedInstanceState) {
8         super.onCreate(savedInstanceState);
9         setContentView(R.layout.activity_main);
10
11         // Initialisation de l'interface
12         textView = findViewById(R.id.textview);
13
14         // Récupération des données sauvegardées
15         if (savedInstanceState != null) {
16             String savedText = savedInstanceState.getString("text");
17             textView.setText(savedText);
18         }
19
20         Log.d(TAG, "onCreate appel ");
21     }
```

```
22
23     @Override
24     protected void onStart() {
25         super.onStart();
26         Log.d(TAG, "onStart appel ");
27     }
28
29     @Override
30     protected void onResume() {
31         super.onResume();
32         Log.d(TAG, "onResume appel ");
33         // Reprendre les animations, la cam ra , etc.
34     }
35
36     @Override
37     protected void onPause() {
38         super.onPause();
39         Log.d(TAG, "onPause appel ");
40         // Sauvegarder les donn es temporaires
41     }
42
43     @Override
44     protected void onStop() {
45         super.onStop();
46         Log.d(TAG, "onStop appel ");
47         // Lib rer les ressources
48     }
49
50     @Override
51     protected void onRestart() {
52         super.onRestart();
53         Log.d(TAG, "onRestart appel ");
54     }
55
56     @Override
57     protected void onDestroy() {
58         super.onDestroy();
59         Log.d(TAG, "onDestroy appel ");
60     }
61
62     // Sauvegarder l' tat lors de la rotation
63     @Override
64     protected void onSaveInstanceState(Bundle outState) {
65         super.onSaveInstanceState(outState);
66         outState.putString("text", textView.getText().toString());
67     }
68 }
```

[TIP] : Utilisez `onSaveInstanceState()` pour sauvegarder l'état lors de la rotation d'écran.

[ERREUR COURANTE] : Oublier d'appeler `super.onXXX()` dans les méthodes du cycle de vie peut causer des crashes.

Configuration Changes

Lors d'un changement de configuration (rotation, changement de langue), Android détruit et recrée l'Activity.

Solutions :

- Utiliser `onSaveInstanceState()` et `onRestoreInstanceState()`
- Utiliser `ViewModel` (architecture moderne)
- Empêcher la recréation avec `android:configChanges` (non recommandé)

Chapitre 3 : Interface Utilisateur sous Android (Partie 1)

Introduction aux Vues et Contrôles

Concepts fondamentaux

Vue (View) : Chaque élément visible et interactif sur l'écran d'une application Android est une instance de la classe `View` ou d'une de ses sous-classes. Que ce soit un bouton, un champ de texte ou une image, tout est une "Vue".

ViewGroup : Permet de regrouper des éléments graphiques. Des `ViewGroup` particuliers sont prédéfinis : ce sont des gabarits (layout) qui proposent une prédisposition des objets graphiques.

Déclaration XML : Les déclarations se font principalement en XML, ce qui évite de passer par les instanciations Java. Cela sépare la présentation de la logique.

Contrôles basiques

TextView Le contrôle le plus simple pour afficher du texte statique. Il est non éditable par l'utilisateur.

Exemple XML :

```
1 <TextView
2     android:id="@+id/textView"
3     android:layout_width="wrap_content"
4     android:layout_height="wrap_content"
5     android:text="Bonjour Android"
6     android:textSize="18sp"
7     android:textColor="#000000"
8     android:gravity="center" />
```

Attributs importants :

- `android:text` : Le contenu textuel
- `android:textSize` : La taille du texte (utiliser `sp` pour le texte)
- `android:textColor` : La couleur du texte
- `android:gravity` : Alignement du texte dans le `TextView`

EditText Permet aux utilisateurs de saisir du texte, essentiel pour les formulaires et les champs de recherche.

Exemple XML :

```
1 <EditText
2     android:id="@+id/editText"
3     android:layout_width="match_parent"
4     android:layout_height="wrap_content"
5     android:hint="Entrez votre nom"
6     android:inputType="textPersonName"
7     android:maxLength="1" />
```

Attributs importants :

- `android:hint` : Texte d'indication affiché si le champ est vide

- `android:inputType` : Spécifie le type de données attendu
 - `text` : Texte normal
 - `textPassword` : Mot de passe (masqué)
 - `number` : Nombre
 - `phone` : Numéro de téléphone
 - `textEmailAddress` : Adresse email

— `android:maxLines` : Nombre maximum de lignes

[TIP] : Utiliser le bon `inputType` améliore l'expérience utilisateur (clavier adapté).

Button Contrôle interactif qui déclenche une action lorsqu'il est pressé.

Exemple XML :

```
1 <Button
2     android:id="@+id/button"
3     android:layout_width="wrap_content"
4     android:layout_height="wrap_content"
5     android:text="Cliquez-moi"
6     android:onClick="onButtonClick" />
```

[TIP] : `android:onClick` permet de lier directement une méthode dans l'Activity, mais il est préférable d'utiliser `setOnClickListener()` dans le code Java pour plus de flexibilité.

Contrôles de sélection

CheckBox Choix multiples. Permet à l'utilisateur de sélectionner ou de désélectionner une ou plusieurs options indépendamment.

Exemple XML :

```
1 <CheckBox
2     android:id="@+id/checkbox"
3     android:layout_width="wrap_content"
4     android:layout_height="wrap_content"
5     android:text="J'accepte les conditions"
6     android:checked="false" />
```

Utilisation Java :

```
1 CheckBox checkBox = findViewById(R.id.checkbox);
2 if (checkBox.isChecked()) {
3     // La case est cochée
4 }
```

RadioGroup et RadioButton Choix unique. Le `RadioGroup` regroupe des `RadioButton` pour assurer un choix exclusif parmi un ensemble d'options.

Exemple XML :

```
1 <RadioGroup
2     android:id="@+id/radioGroup"
3     android:layout_width="wrap_content"
4     android:layout_height="wrap_content"
5     android:orientation="vertical">
6
7     <RadioButton
8         android:id="@+id/radioOption1"
9         android:layout_width="wrap_content"
```

```

10     android:layout_height="wrap_content"
11     android:text="Option 1" />
12
13     <RadioButton
14         android:id="@+id/radioOption2"
15         android:layout_width="wrap_content"
16         android:layout_height="wrap_content"
17         android:text="Option 2" />
18 </RadioGroup>

```

[IMPORTANT] : Les `RadioButton` doivent être contenus dans un `RadioGroup` pour fonctionner correctement.

Positionnement et Layouts

Types de Layouts

LinearLayout Organisation simple des vues en ligne (horizontale ou verticale). Idéal pour les listes et les arrangements linéaires simples.

Exemple complet :

```

1 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
2     android:orientation="vertical"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent"
5     android:gravity="center"
6     android:padding="16dp"
7     android:id="@+id/accueilid">
8
9     <TextView
10         android:layout_width="wrap_content"
11         android:layout_height="wrap_content"
12         android:text="Bienvenue"
13         android:textSize="24sp" />
14
15     <Button
16         android:layout_width="wrap_content"
17         android:layout_height="wrap_content"
18         android:text="Commencer" />
19 </LinearLayout>

```

Attributs clés :

- `android:orientation` : "vertical" ou "horizontal"
- `android:gravity` : Alignement des enfants
- `android:layout_weight` : Pour les enfants, permet de distribuer l'espace

ConstraintLayout Positionnement flexible basé sur des contraintes relatives. Recommandé pour toutes les interfaces complexes et performantes.

Exemple :

```

1 <androidx.constraintlayout.widget.ConstraintLayout
2     xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent">
6
7     <Button

```

```

8      android:id="@+id/button"
9      android:layout_width="wrap_content"
10     android:layout_height="wrap_content"
11     android:text="Bouton"
12     app:layout_constraintTop_toTopOf="parent"
13     app:layout_constraintStart_toStartOf="parent"
14     app:layout_constraintEnd_toEndOf="parent" />
15 </androidx.constraintlayout.widget.ConstraintLayout>

```

[**TIP**] : ConstraintLayout est plus performant et flexible que RelativeLayout. Utilisez-le pour les nouvelles applications.

FrameLayout Utilisé pour afficher un seul élément ou superposer plusieurs vues. Idéal pour les fragments ou les superpositions d'éléments.

TableLayout Organisation des vues en lignes et colonnes, similaire à une table HTML. Convient pour l'affichage de données tabulaires.

RelativeLayout Positionnement relatif. Permet de positionner chaque élément enfant par rapport aux bords de son conteneur parent ou par rapport à d'autres vues.

[**NOTE**] : RelativeLayout est déprécié au profit de ConstraintLayout.

Attributs des Layouts

Layout Width et Height :

- "match_parent" : L'élément remplit tout l'espace disponible de son parent
- "wrap_content" : L'élément prend la taille minimale nécessaire pour contenir son contenu
- "XXdp" : Taille fixe en dp (density-independent pixels)

[**TIP**] : Toujours utiliser dp pour les dimensions, jamais px.

Identificateurs (ID) Android

- android:id="@+id/idElement" : Utilisation lors de la déclaration d'un nouvel élément dans le XML
- @id/idElement : Référence à un élément existant pour le positionnement ou la manipulation

[**ERREUR COURANTE**] : Oublier le + dans @+id/ lors de la première déclaration.

Chapitre 3 : Interface Utilisateur (Partie 2) - TRÈS IMPORTANT

[**ATTENTION**] : Cette partie est cruciale pour l'examen. Une attention particulière doit être portée aux notions à partir de la deuxième moitié du chapitre 3.

Élasticité et Adaptabilité des UI

Unités de mesure

dp (density-independent pixel) : Le dp est une unité virtuelle qui s'adapte automatiquement à la densité de l'écran. Il est l'unité préférée pour spécifier les tailles des éléments, les marges et le padding afin d'assurer une apparence uniforme sur différents appareils.

sp (scale-independent pixel) : Similaire au dp, le sp s'ajuste non seulement à la densité de l'écran mais aussi aux préférences de taille de texte de l'utilisateur (paramètres d'accessibilité). Il est donc essentiel pour les tailles de texte.

px (pixel) : Le px représente un pixel réel de l'écran. Sa taille physique varie énormément selon la densité de l'appareil, rendant les mises en page incohérentes. Il est fortement déconseillé pour la conception d'interfaces utilisateur adaptatives.

[REGLE D'OR] :

- Utiliser **dp** pour les dimensions (largeur, hauteur, marges, padding)
- Utiliser **sp** pour les tailles de texte
- **JAMAIS** utiliser **px** sauf cas très spécifiques

Bonnes pratiques UI adaptatives

1. Toujours utiliser **dp** et **sp**
2. Éviter les tailles fixes en pixels (px)
3. Utiliser **ConstraintLayout** pour s'adapter aux écrans
4. Créer des ressources alternatives :
 - `/layout-sw600dp/` pour les tablettes
 - `/layout-land/` pour le mode paysage
 - `/values-sw600dp/` pour les dimensions de tablettes
5. Tester avec l'émulateur sur différentes résolutions
6. Utiliser **ScrollView** pour les contenus longs

ConstraintLayout - Approfondissement

Le **ConstraintLayout** est un gabarit de mise en page moderne et flexible qui permet de créer des interfaces utilisateur responsives et fluides. Il utilise un système de contraintes pour positionner et dimensionner les vues.

Types de Contraintes

Contraintes Relatives : Positionnent une vue par rapport aux bords ou aux lignes de base d'une autre vue.

Exemple complet :

```

1 <androidx.constraintlayout.widget.ConstraintLayout
2     xmlns:android="http://schemas.android.com/apk/res/android"
3     xmlns:app="http://schemas.android.com/apk/res-auto"
4     android:layout_width="match_parent"
5     android:layout_height="match_parent">
6
7     <TextView
8         android:id="@+id/textView1"
9         android:layout_width="wrap_content"
10        android:layout_height="wrap_content"
11        android:text="Texte 1"
12        app:layout_constraintTop_toTopOf="parent"
13        app:layout_constraintStart_toStartOf="parent"
14        android:layout_marginTop="16dp"
15        android:layout_marginStart="16dp" />
16
17    <TextView
18        android:id="@+id/textView2"

```

```

19     android:layout_width="wrap_content"
20     android:layout_height="wrap_content"
21     android:text="Texte 2"
22     app:layout_constraintTop_toBottomOf="@id/textView1"
23     app:layout_constraintStart_toStartOf="parent"
24     android:layout_marginTop="8dp"
25     app:layout_constraintEnd_toEndOf="parent" />
26 </androidx.constraintlayout.widget.ConstraintLayout>

```

Attributs de contraintes courants :

- `layout_constraintStart_toStartOf` : Aligne le début avec le début de la cible
- `layout_constraintTop_toTopOf` : Aligne le haut avec le haut de la cible
- `layout_constraintBaseline_toBaselineOf` : Aligne les lignes de base de texte
- `layout_constraintEnd_toEndOf` : Aligne la fin avec la fin de la cible
- `layout_constraintBottom_toBottomOf` : Aligne le bas avec le bas de la cible

Contraintes Avancées :

- **Contraintes Circulaires** : `layout_constraintCircle`, `layout_constraintCircleRadius`, `layout_constraintCircleAngle`
- **Chaînes (Chains)** : `layout_constraintHorizontal_chainStyle` (spread, spread_inside, packed)
- **Barrières (Barriers)** : Créent une ligne de référence virtuelle basée sur plusieurs vues
- **Guidelines** : Lignes de référence invisibles pour l'alignement

Listes et Adaptateurs

Définition

Les **Adaptateurs** font le lien entre une source de données et une vue (liste, grille, etc.), permettant d'afficher des données complexes de manière efficace dans des composants d'interface utilisateur.

Principe du pattern Adapter

Permet à des interfaces incompatibles de collaborer. Il s'interpose entre les données brutes et le composant UI, adaptant les données pour l'affichage.

Schéma : Données → Adapter → ListView/RecyclerView

Types courants

- **ArrayAdapter** : Pour les données simples comme des chaînes de caractères
- **BaseAdapter** : Pour les vues personnalisées
- **RecyclerView.Adapter** : Pour des listes plus performantes et flexibles

Exemple avec ListView et ArrayAdapter

XML Layout :

```

1 <ListView
2     android:id="@+id/listView"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent" />

```

Java Code :

```

1 ListView listView = findViewById(R.id.listView);
2 String[] villes = {"Rabat", "Agadir", "Safi", "Casablanca", "Tanger"};
3
4 ArrayAdapter<String> adapter = new ArrayAdapter<>(
5     this, // Context
6     android.R.layout.simple_list_item_1, // Layout pour chaque item
7     villes // Donn es
8 );
9
10 listView.setAdapter(adapter);
11
12 // Gestion du clic sur un lment
13 listView.setOnItemClickListener((parent, view, position, id) -> {
14     String villeSelectionnee = villes[position];
15     Toast.makeText(this, "Ville s lectionn e: " + villeSelectionnee,
16         Toast.LENGTH_SHORT).show();
17 });

```

[EXPLICATION] :

- ArrayAdapter prend 3 paramètres : Context, layout, données
- android.R.layout.simple_list_item_1 est un layout prédéfini Android
- setAdapter() lie l'adapter à la ListView
- setOnItemClickListener() gère les clics sur les éléments

RecyclerView - Approfondissement

Le RecyclerView est la version moderne et plus performante que ListView pour l'affichage de grandes collections de données. Il offre une meilleure gestion de la mémoire et une fluidité accrue grâce à son architecture modulaire.

Composants du RecyclerView :

- **Adapter** : Définit la structure des éléments et assure la liaison entre les données et les vues
- **ViewHolder** : Réutilise les vues existantes pour un élément de liste, optimisant ainsi les performances de défilement
- **LayoutManager** : Définit la manière dont les éléments sont disposés (liste linéaire, grille ou cascade)

Exemple RecyclerView complet

1. Layout XML pour un item (item_layout.xml) :

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
3     android:layout_width="match_parent"
4     android:layout_height="wrap_content"
5     android:orientation="horizontal"
6     android:padding="16dp">
7
8     <TextView
9         android:id="@+id/item_text_view"
10        android:layout_width="0dp"
11        android:layout_height="wrap_content"
12        android:layout_weight="1"
13        android:textSize="16sp" />

```

```

14     <ImageView
15         android:id="@+id/item_icon"
16         android:layout_width="48dp"
17         android:layout_height="48dp"
18         android:src="@android:drawable/ic_menu_more" />
19 </LinearLayout>
20

```

2. Layout XML principal avec RecyclerView :

```

1 <androidx.recyclerview.widget.RecyclerView
2     android:id="@+id/my_recycler_view"
3     android:layout_width="match_parent"
4     android:layout_height="match_parent" />

```

3. Classe ViewHolder :

```

1 public static class MonViewHolder extends RecyclerView.ViewHolder {
2     public TextView textView;
3     public ImageView imageView;
4
5     public MonViewHolder(@NonNull View itemView) {
6         super(itemView);
7         textView = itemView.findViewById(R.id.item_text_view);
8         imageView = itemView.findViewById(R.id.item_icon);
9     }
10 }

```

4. Classe Adapter complète :

```

1 public class MonAdapter extends RecyclerView.Adapter<MonAdapter.
2     MonViewHolder> {
3     private String[] donnees;
4
5     public MonAdapter(String[] donnees) {
6         this.donnees = donnees;
7     }
8
9     @NonNull
10    @Override
11    public MonViewHolder onCreateViewHolder(@NonNull ViewGroup parent,
12        int viewType) {
13        // Cr er une nouvelle vue
14        View view = LayoutInflater.from(parent.getContext())
15            .inflate(R.layout.item_layout, parent, false);
16        return new MonViewHolder(view);
17    }
18
19    @Override
20    public void onBindViewHolder(@NonNull MonViewHolder holder, int
21        position) {
22        // Remplir la vue avec les donn es la position donn e
23        holder.textView.setText(donnees[position]);
24
25        // Gestion du clic
26        holder.itemView.setOnClickListener(v -> {
27            Toast.makeText(v.getContext(),
28                "Clic sur: " + donnees[position],
29                Toast.LENGTH_SHORT).show();
30        });
31    }
32 }

```



```

28     }
29
30     @Override
31     public int getItemCount() {
32         return donnees.length;
33     }
34
35     // ViewHolder d fini ci-dessus
36     public static class MonViewHolder extends RecyclerView.ViewHolder {
37         // ... (code du ViewHolder)
38     }
39 }

```

5. Utilisation dans l'Activity :

```

1 RecyclerView recyclerView = findViewById(R.id.my_recycler_view);
2 recyclerView.setLayoutManager(new LinearLayoutManager(this));
3 recyclerView.setAdapter(new MonAdapter(vosDonnees));

```

[EXPLICATION DETAILLEE] :

- `onCreateViewHolder()` : Crée une nouvelle vue (appelée seulement quand nécessaire)
- `onBindViewHolder()` : Remplit une vue existante avec de nouvelles données (appelée pour chaque élément visible)
- `getItemCount()` : Retourne le nombre total d'éléments
- `ViewHolder` : Réutilise les vues, améliorant les performances
- `LayoutManager` : Détermine comment les éléments sont disposés

[ATTENTION] Erreurs courantes :

- Oublier de définir le `LayoutManager`
- Ne pas implémenter toutes les méthodes requises
- Oublier le `@NonNull` sur les paramètres

Navigation entre Activités

Intent

Un **Intent** est un message abstrait qui permet de démarrer une autre activité, un service, ou de diffuser un événement. Il agit comme un messenger entre les composants de l'application ou même entre différentes applications. Il peut transporter des données entre les activités.

Types d'Intents :

- **Intent Explicite** : Spécifie exactement quelle Activity démarrer (classe connue)
- **Intent Implicite** : Décrit l'action à accomplir, Android choisit l'application appropriée

Navigation simple

```

1 // Cr er un Intent explicite
2 Intent intent = new Intent(this, SecondActivity.class);
3 startActivity(intent);

```

[EXPLICATION] : `this` est le Context actuel, `SecondActivity.class` est la classe de destination.

Transfert de données

Envoi de données :

```

1 Intent intent = new Intent(this, SecondActivity.class);
2 intent.putExtra("nom", "Ali");
3 intent.putExtra("age", 25);
4 intent.putExtra("estEtudiant", true);
5 startActivity(intent);

```

Récupération de données :

```

1 // Dans SecondActivity
2 Intent intent = getIntent();
3 String nom = intent.getStringExtra("nom");
4 int age = intent.getIntExtra("age", 0); // 0 est la valeur par d faut
5 boolean estEtudiant = intent.getBooleanExtra("estEtudiant", false);

```

[TIP] : Toujours vérifier si les données existent avant de les utiliser :

```

1 if (intent.hasExtra("nom")) {
2     String nom = intent.getStringExtra("nom");
3 }

```

Gestion des Événements

OnClickListener

Le gestionnaire d'événements le plus fondamental en Android est l'OnClickListener, utilisé pour détecter les clics des utilisateurs sur divers composants d'interface utilisateur.

Méthode 1 : Dans le XML :

```

1 <Button
2     android:id="@+id/mon_bouton"
3     android:onClick="onButtonClick"
4     android:text="Cliquez" />

```

```

1 public void onButtonClick(View v) {
2     Toast.makeText(this, "Bouton cliqué !", Toast.LENGTH_SHORT).show();
3 }

```

Méthode 2 : Dans le code Java (recommandé) :

```

1 Button monBouton = findViewById(R.id.mon_bouton);
2 monBouton.setOnClickListener(new View.OnClickListener() {
3     @Override
4     public void onClick(View v) {
5         Toast.makeText(MainActivity.this, "Bouton cliqué !",
6             Toast.LENGTH_SHORT).show();
7     }
8 });

```

Méthode 3 : Lambda (Java 8+) :

```

1 Button monBouton = findViewById(R.id.mon_bouton);
2 monBouton.setOnClickListener(v -> {
3     Toast.makeText(this, "Bouton cliqué !", Toast.LENGTH_SHORT).show();
4 });

```

[TIP] : Utiliser les lambdas rend le code plus concis et lisible.

OnFocusChangeListener

Les événements de focus sont essentiels pour gérer l'interaction des utilisateurs avec des champs de saisie de texte.

```

1 EditText monChampTexte = findViewById(R.id.mon_champ_texte);
2 monChampTexte.setOnFocusChangeListener(new View.OnFocusChangeListener()
3 {
4     @Override
5     public void onFocusChange(View v, boolean hasFocus) {
6         if (hasFocus) {
7             // Le composant a gagn le focus
8             // Exemple : changer la couleur de fond
9             v.setBackgroundColor(Color.LIGHT_GRAY);
10        } else {
11            // Le composant a perdu le focus
12            // Validation possible ici
13            v.setBackgroundColor(Color.WHITE);
14            String texte = ((EditText) v).getText().toString();
15            if (texte.isEmpty()) {
16                Toast.makeText(MainActivity.this,
17                    "Le champ ne peut pas tre vide",
18                    Toast.LENGTH_SHORT).show();
19            }
20        }
21    });

```

TextWatcher

Pour détecter et réagir aux changements dans un champ de saisie de texte (EditText).

```

1 EditText monEditText = findViewById(R.id.mon_edit_text);
2 monEditText.addTextChangedListener(new TextWatcher() {
3     @Override
4     public void beforeTextChanged(CharSequence s, int start, int count,
5         int after) {
6         // Avant que le texte change
7     }
8
9     @Override
10    public void onTextChanged(CharSequence s, int start, int before, int
11        count) {
12        // Pendant que le texte change
13        Log.d("TextWatcher", "Pendant modification: " + s.toString());
14
15        // Exemple : validation en temps r el
16        if (s.length() < 3) {
17            monEditText.setError("Minimum 3 caract res");
18        } else {
19            monEditText.setError(null);
20        }
21    }
22
23    @Override
24    public void afterTextChanged(Editable s) {
25        // Apr s que le texte a chang
26        // Exemple : mise jour d'un compteur
27        TextView compteur = findViewById(R.id.compteur);

```

```

26     compteur.setText(s.length() + " caract res");
27 }
28 });

```

[EXPLICATION] :

- `beforeTextChanged` : Appelé avant que le texte change
- `onTextChanged` : Appelé pendant le changement (le plus utilisé)
- `afterTextChanged` : Appelé après le changement

Les Menus

Menus d'options

Les menus d'options sont définis dans des fichiers XML stockés dans le dossier `/res/menu/`.

Fichier menu (`res/menu/main_menu.xml`) :

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <menu xmlns:android="http://schemas.android.com/apk/res/android">
3     <item
4         android:id="@+id/action_settings"
5         android:title="Param tres"
6         android:icon="@android:drawable/ic_menu_preferences" />
7
8     <item
9         android:id="@+id/action_help"
10        android:title="Aide" />
11 </menu>

```

Code Java :

```

1 @Override
2 public boolean onCreateOptionsMenu(Menu menu) {
3     getMenuInflater().inflate(R.menu.main_menu, menu);
4     return true; // Afficher le menu
5 }
6
7 @Override
8 public boolean onOptionsItemSelected(MenuItem item) {
9     switch (item.getItemId()) {
10        case R.id.action_settings:
11            Toast.makeText(this, "Param tres", Toast.LENGTH_SHORT).show();
12            return true;
13        case R.id.action_help:
14            Toast.makeText(this, "Aide", Toast.LENGTH_SHORT).show();
15            return true;
16        default:
17            return super.onOptionsItemSelected(item);
18    }
19 }

```

Menus contextuels

Les menus contextuels apparaissent lorsque l'utilisateur effectue un clic long sur un élément de l'interface.

Fichier menu contextuel (`res/menu/context_menu.xml`) :

```

1 <menu xmlns:android="http://schemas.android.com/apk/res/android">
2   <item
3       android:id="@+id/action_edit"
4       android:title="Modifieur" />
5   <item
6       android:id="@+id/action_delete"
7       android:title="Supprimeur" />
8 </menu>

```

Code Java :

```

1 TextView myTextView = findViewById(R.id.my_text_view);
2 registerForContextMenu(myTextView);
3
4 @Override
5 public void onCreateContextMenu(ContextMenu menu, View v,
6     ContextMenu.ContextMenuInfo menuInfo) {
7     super.onCreateContextMenu(menu, v, menuInfo);
8     if (v.getId() == R.id.my_text_view) {
9         getMenuInflater().inflate(R.menu.context_menu, menu);
10    }
11 }
12
13 @Override
14 public boolean onOptionsItemSelected(MenuItem item) {
15     switch (item.getItemId()) {
16         case R.id.action_edit:
17             Toast.makeText(this, "Action Modifieur",
18                 Toast.LENGTH_SHORT).show();
19             return true;
20         case R.id.action_delete:
21             Toast.makeText(this, "Action Supprimeur",
22                 Toast.LENGTH_SHORT).show();
23             return true;
24         default:
25             return super.onOptionsItemSelected(item);
26     }
27 }

```

[EXPLICATION] : `registerForContextMenu()` enregistre une vue pour afficher un menu contextuel lors d'un clic long.

Dialogue avec l'Utilisateur

Toast

Le Toast est le moyen le plus simple d'afficher un message bref à l'utilisateur.

```

1 Toast.makeText(this, "This is a Toast Message",
2     Toast.LENGTH_SHORT).show();
3
4 // Ou avec une durée personnalisée
5 Toast.makeText(this, "Message long", Toast.LENGTH_LONG).show();

```

[TIP] : `LENGTH_SHORT` = 2 secondes, `LENGTH_LONG` = 3.5 secondes.

AlertDialog

L'`AlertDialog` est un composant puissant pour créer des boîtes de dialogue modales.

Exemple simple :

```

1 new AlertDialog.Builder(this)
2   .setTitle("Confirmation")
3   .setMessage("Voulez-vous quitter ?")
4   .setPositiveButton("Oui", (d, i) -> finish())
5   .setNegativeButton("Non", null)
6   .show();

```

Exemple avec choix multiples :

```

1 String[] options = {"Option 1", "Option 2", "Option 3"};
2 new AlertDialog.Builder(this)
3   .setTitle("Choisissez une option")
4   .setItems(options, (dialog, which) -> {
5       Toast.makeText(this, "Choix: " + options[which],
6           Toast.LENGTH_SHORT).show();
7   })
8   .show();

```

Exemple avec EditText :

```

1 EditText input = new EditText(this);
2 input.setInputType(InputType.TYPE_CLASS_TEXT);
3 new AlertDialog.Builder(this)
4   .setTitle("Entrez votre nom")
5   .setView(input)
6   .setPositiveButton("OK", (d, i) -> {
7       String nom = input.getText().toString();
8       Toast.makeText(this, "Bonjour " + nom,
9           Toast.LENGTH_SHORT).show();
10  })
11  .setNegativeButton("Annuler", null)
12  .show();

```

Snackbar

Les Snackbars sont la solution moderne recommandée par Google pour afficher des messages temporaires.

```

1 Snackbar.make(findViewById(R.id.main), "Données enregistrées",
2   Snackbar.LENGTH_SHORT)
3   .setAction("ANNULER", v -> {
4       // Action effectuer
5   })
6   .show();

```

[TIP] : Snackbar nécessite la dépendance Material Design. Il est plus moderne que Toast.

Chapitre 4 : Persistance des Données

Introduction

Comprendre comment une application Android peut conserver ses données au-delà d'une seule exécution, garantissant la disponibilité et l'intégrité des informations même après la fermeture de l'application ou le redémarrage de l'appareil.

Options de persistance :

1. **SharedPreferences** : Pour les préférences utilisateur (clé-valeur)

2. **Fichiers** : Stockage interne ou externe
3. **SQLite** : Base de données relationnelle
4. **Room** : Bibliothèque moderne pour SQLite (recommandée)

Accès aux Fichiers

Types de stockage

Stockage Interne : Cet espace est propre à l'application, ce qui signifie que les données y sont isolées et non accessibles par d'autres applications. Il est fortement recommandé pour le stockage des informations confidentielles ou des données essentielles au fonctionnement exclusif de votre application.

Stockage Externe : Ce stockage est partagé avec d'autres applications et est donc adapté aux fichiers volumineux ou aux contenus que l'utilisateur souhaite partager (photos, documents). L'accès à cet espace nécessite la gestion appropriée des permissions utilisateur.

Écriture dans un fichier interne

```
1 String filename = "mon_fichier.txt";
2 String contenu = "Contenu        crire ";
3
4 try (FileOutputStream fos = openFileOutput(filename, Context.
    MODE_PRIVATE)) {
5     fos.write(contenu.getBytes());
6     Toast.makeText(this, "Fichier sauvegard ", Toast.LENGTH_SHORT).show
        ();
7 } catch (IOException e) {
8     e.printStackTrace();
9 }
```

[EXPLICATION] :

- `openFileOutput()` crée un fichier dans le répertoire interne de l'app
- `MODE_PRIVATE` : Fichier accessible uniquement par l'application
- `try-with-resources` ferme automatiquement le flux

Lecture d'un fichier interne

```
1 String filename = "mon_fichier.txt";
2 String contenu = "";
3
4 try (FileInputStream fis = openFileInput(filename)) {
5     byte[] buffer = new byte[fis.available()];
6     fis.read(buffer);
7     contenu = new String(buffer);
8 } catch (IOException e) {
9     e.printStackTrace();
10 }
11
12 TextView textView = findViewById(R.id.textView);
13 textView.setText(contenu);
```

Bonnes pratiques pour la gestion des fichiers

1. **Vérification d'existence** : Toujours vérifier la présence d'un fichier avant toute tentative de lecture
2. **Fermeture des flux** : Il est crucial de fermer systématiquement tous les flux (`InputStream`, `OutputStream`) après usage (utiliser `try-with-resources`)
3. **Protection des données sensibles** : Pour les informations confidentielles, utilisez le stockage interne
4. **Gestion des erreurs** : Mettez en place des blocs `try-catch` robustes
5. **Maintenance des fichiers** : Prévoyez des mécanismes pour la suppression ou la mise à jour régulière des fichiers obsolètes

SharedPreferences

Les `SharedPreferences` sont un mécanisme simple pour stocker des données légères sous forme de paires clé-valeur, utilisées principalement pour les préférences utilisateur et les réglages d'application.

Obtenir l'Instance

```
1 // Méthode 1 : Fichier de préférences nommé
2 SharedPreferences prefs = getSharedPreferences("MonAppPrefs",
3     Context.MODE_PRIVATE);
4
5 // Méthode 2 : Préférences par défaut
6 SharedPreferences prefs = PreferenceManager
7     .getDefaultSharedPreferences(this);
```

[TIP] : Utiliser un nom unique pour éviter les conflits avec d'autres apps.

Lire des Données

```
1 String nom = prefs.getString("nomUtilisateur", "Invité"); // "Invité "
2     est la valeur par défaut
3 int age = prefs.getInt("age", 0);
4 boolean notifs = prefs.getBoolean("notificationsActives", false);
5 float score = prefs.getFloat("score", 0.0f);
6 long timestamp = prefs.getLong("derniereConnexion", 0);
```

[IMPORTANT] : Toujours fournir une valeur par défaut lors de la lecture.

Écrire des Données

```
1 SharedPreferences.Editor editor = prefs.edit();
2 editor.putString("nomUtilisateur", "John Doe");
3 editor.putInt("age", 30);
4 editor.putBoolean("notificationsActives", true);
5 editor.putFloat("score", 95.5f);
6 editor.putLong("timestamp", System.currentTimeMillis());
7 editor.apply(); // ou editor.commit()
```

[TIP] : Utiliser `apply()` plutôt que `commit()` car il est asynchrone et plus performant.

Supprimer des Données

```
1 editor.remove("nomUtilisateur");
2 editor.apply();
3
4 // Ou supprimer toutes les pr f rences
5 editor.clear();
6 editor.apply();
```

apply() vs commit()

- **apply()** : Asynchrone, plus rapide, pas de valeur de retour, recommandé
- **commit()** : Synchrone, bloque le thread, retourne un booléen (succès/échec)

Recommandation : Utiliser `apply()` dans la plupart des cas, sauf si vous avez besoin de savoir immédiatement si l'opération a réussi.

Bases de Données SQLite

SQLite est une solution de base de données relationnelle légère et efficace, idéale pour la persistance des données directement sur l'appareil Android.

Caractéristiques

- **Base de données Embarquée** : SQLite est entièrement intégrée à l'application
- **Stockage Structuré** : Permet d'organiser et de stocker des données dans des tables
- **Autonome et Locale** : Absence de serveur, fonctionnement hors ligne complet
- **Compatible SQL** : Supporte la majorité des commandes SQL standards

SQLiteOpenHelper

SQLiteOpenHelper est une classe utilitaire fournie par Android pour faciliter la création, l'ouverture et la gestion des versions de votre base de données.

Création d'une classe DatabaseHelper

```
1 public class DatabaseHelper extends SQLiteOpenHelper {
2     public static final String DATABASE_NAME = "MaBaseDeDonnees.db";
3     public static final int DATABASE_VERSION = 1;
4
5     // Nom de la table
6     public static final String TABLE_PRODUITS = "produits";
7
8     // Colonnes de la table
9     public static final String COL_ID = "id";
10    public static final String COL_NOM = "nom";
11    public static final String COL_PRIX = "prix";
12    public static final String COL_QUANTITE = "quantite";
13
14    public DatabaseHelper(Context context) {
15        super(context, DATABASE_NAME, null, DATABASE_VERSION);
16    }
17
18    @Override
19    public void onCreate(SQLiteDatabase db) {
```

```

20     String CREATE_TABLE = "CREATE TABLE " + TABLE_PRODUITS + " (" +
21                             COL_ID + " INTEGER PRIMARY KEY
22                                 AUTOINCREMENT," +
23                             COL_NOM + " TEXT," +
24                             COL_PRIX + " REAL," +
25                             COL_QUANTITE + " INTEGER)";
26     db.execSQL(CREATE_TABLE);
27
28     @Override
29     public void onUpgrade(SQLiteDatabase db, int oldVersion, int
30         newVersion) {
31         db.execSQL("DROP TABLE IF EXISTS " + TABLE_PRODUITS);
32         onCreate(db);
33     }

```

[EXPLICATION] :

- onCreate() : Appelée lors de la première création de la base
- onUpgrade() : Appelée quand la version change
- AUTOINCREMENT : Génère automatiquement un ID unique

Opérations CRUD

Insertion de Données

```

1 public boolean insertData(String nom, double prix, int quantite) {
2     SQLiteDatabase db = this.getWritableDatabase();
3     ContentValues contentValues = new ContentValues();
4     contentValues.put(COL_NOM, nom);
5     contentValues.put(COL_PRIX, prix);
6     contentValues.put(COL_QUANTITE, quantite);
7     long result = db.insert(TABLE_PRODUITS, null, contentValues);
8     return result != -1; // -1 signifie erreur
9 }

```

[EXPLICATION] : ContentValues est un conteneur pour les paires clé-valeur, similaire à un Bundle.

Lecture de Données Méthode 1 : rawQuery :

```

1 public Cursor getData() {
2     SQLiteDatabase db = this.getWritableDatabase();
3     Cursor res = db.rawQuery("SELECT * FROM " + TABLE_PRODUITS, null);
4     return res;
5 }

```

Méthode 2 : query (recommandée) :

```

1 public Cursor getAllProduits() {
2     SQLiteDatabase db = this.getReadableDatabase();
3     return db.query(TABLE_PRODUITS,
4         null,           // colonnes (null = toutes)
5         null,           // WHERE clause
6         null,           // arguments WHERE
7         null,           // GROUP BY
8         null,           // HAVING
9         COL_NOM + " ASC"); // ORDER BY
10 }

```

Utilisation du Cursor :

```

1 Cursor cursor = myDb.getAllProduits();
2 if (cursor.moveToFirst()) {
3     do {
4         int id = cursor.getInt(cursor.getColumnIndex(COL_ID));
5         String nom = cursor.getString(cursor.getColumnIndex(COL_NOM));
6         double prix = cursor.getDouble(cursor.getColumnIndex(COL_PRIX));
7         int quantite = cursor.getInt(cursor.getColumnIndex(COL_QUANTITE));
8
9         Log.d("DB", "ID: " + id + ", Nom: " + nom);
10    } while (cursor.moveToNext());
11 }
12 cursor.close(); // IMPORTANT : fermer le cursor

```

[ERREUR COURANTE] : Oublier de fermer le Cursor peut causer des fuites mémoire.

Mise à Jour de Données

```

1 public boolean updateData(String id, String nom, double prix, int
   quantite) {
2     SQLiteDatabase db = this.getWritableDatabase();
3     ContentValues contentValues = new ContentValues();
4     contentValues.put(COL_NOM, nom);
5     contentValues.put(COL_PRIX, prix);
6     contentValues.put(COL_QUANTITE, quantite);
7
8     int result = db.update(TABLE_PRODUITS,
9         contentValues,
10        COL_ID + " = ?",
11        new String[]{id});
12     return result > 0;
13 }

```

[EXPLICATION] : Le ? est un placeholder remplacé par les valeurs du tableau `new String[]{id}`.

Suppression de Données

```

1 public Integer deleteData(String id) {
2     SQLiteDatabase db = this.getWritableDatabase();
3     return db.delete(TABLE_PRODUITS, COL_ID + " = ?", new String[]{id});
4 }

```

Utilisation dans une Activity

```

1 public class MainActivity extends AppCompatActivity {
2     DatabaseHelper myDb;
3
4     @Override
5     protected void onCreate(Bundle savedInstanceState) {
6         super.onCreate(savedInstanceState);
7         setContentView(R.layout.activity_main);
8         myDb = new DatabaseHelper(this);
9
10        // Exemple d'insertion
11        boolean isInserted = myDb.insertData("Livre", 19.99, 50);
12        if (isInserted) {
13            Toast.makeText(this, "Donn es ins r es",

```

```

14         Toast.LENGTH_LONG).show();
15     }
16
17     // Exemple de lecture
18     Cursor cursor = myDb.getAllProduits();
19     // ... traiter le cursor
20     cursor.close();
21 }
22 }

```

[TIP] : Créer une instance unique de DatabaseHelper (Singleton) pour éviter les conflits.

Chapitre 5 : Communication et Threads

Thread Principal (UI Thread)

Le thread principal (UI Thread) est responsable de l’affichage de l’interface utilisateur et de la gestion des interactions. Il ne doit jamais être bloqué par des opérations longues.

[ATTENTION] **Règle d’or** : Toutes les opérations qui prennent plus de quelques millisecondes doivent être exécutées sur un thread secondaire.

Opérations à éviter sur le UI Thread :

- Requêtes réseau
- Accès à la base de données (lectures/écritures longues)
- Traitement d’images lourdes
- Calculs intensifs

Multi-threading

Le multi-threading permet d’exécuter des tâches en arrière-plan sans bloquer l’interface utilisateur, garantissant une expérience utilisateur fluide.

Handler

Le Handler permet de communiquer entre threads et d’exécuter du code sur le thread principal depuis un thread secondaire.

Exemple :

```

1 Handler handler = new Handler(Looper.getMainLooper());
2
3 // Dans un thread secondaire
4 new Thread(() -> {
5     // Opération longue
6     String resultat = faireOperationLongue();
7
8     // Mettre à jour l'UI sur le thread principal
9     handler.post(() -> {
10         TextView textView = findViewById(R.id.textView);
11         textView.setText(resultat);
12     });
13 }).start();

```

[EXPLICATION] : `Looper.getMainLooper()` obtient le Looper du thread principal.

AsyncTask (Déprécié)

AsyncTask était une classe qui permettait d'exécuter des opérations en arrière-plan et de publier les résultats sur le thread UI. Elle est maintenant dépréciée depuis Android 11.

[IMPORTANT] : Ne plus utiliser AsyncTask dans les nouvelles applications.

Recommandation moderne : Utiliser Coroutines (Kotlin) ou **ExecutorService** avec **Handler** (Java).

ExecutorService (Recommandé)

Exemple moderne :

```

1 ExecutorService executor = Executors.newSingleThreadExecutor();
2 Handler handler = new Handler(Looper.getMainLooper());
3
4 executor.execute(() -> {
5     // Opération en arrière-plan
6     String resultat = faireOperationLongue();
7
8     // Mettre à jour l'UI
9     handler.post(() -> {
10         updateUI(resultat);
11     });
12 });

```

Chapitre 6 : Intents et Communication entre Composants

Introduction aux Intents

Un **Intent** est un message envoyé à Android pour déclencher une action. Il permet la communication entre :

- Les activités d'une même application
- Les services
- D'autres applications installées
- Des composants du système Android (ex. AlarmManager)

Les Intents Explicites

Permettent de lancer une activité précise (classe connue).

```

1 Intent intent = new Intent(this, DetailActivity.class);
2 startActivity(intent);

```

Utilisés pour la navigation interne à l'application. Chaque activité cible doit être déclarée dans le **AndroidManifest.xml**.

Les Intents Implicites

Décrivent l'action à accomplir, sans nommer le composant. Android choisit automatiquement une application capable de répondre.

```

1 Intent intent = new Intent(Intent.ACTION_VIEW,
2     Uri.parse("https://emsi.ma"));
3 startActivity(intent);

```

Composantes d'un Intent

- **Action** : `intent.setAction(Intent.ACTION_VIEW)`
- **Data (URI)** : `intent.setData(Uri.parse("https://emsi.ma"))`
- **Type** : `intent.setType("text/plain")`
- **Extras** : `intent.putExtra("key", "value")`
- **Category** : `intent.addCategory(Intent.CATEGORY_BROWSABLE)`
- **Flags** : `intent.setFlags(Intent.FLAG_ACTIVITY_NEW_TASK)`

Exemples d'Actions Prédéfinies

- `Intent.ACTION_VIEW` : Ouvrir un lien, une image ou un contact
- `Intent.ACTION_DIAL` : Ouvrir le composeur téléphonique
- `Intent.ACTION_SEND` : Partager du contenu (email par exemple)
- `Intent.ACTION_PICK` : Sélectionner une image ou un contact
- `Intent.ACTION_CALL` : Faire un appel (nécessite permission)

Exemples Pratiques

Ouvrir une Page Web

```
1 Uri webpage = Uri.parse("https://www.google.com");
2 Intent webIntent = new Intent(Intent.ACTION_VIEW, webpage);
3 startActivity(webIntent);
```

Faire un Appel Téléphonique

```
1 Uri number = Uri.parse("tel:0645127859");
2 Intent call = new Intent(Intent.ACTION_DIAL, number);
3 startActivity(call);
```

[NOTE] : `ACTION_DIAL` ouvre le dialer sans appeler. Pour appeler directement, utiliser `ACTION_CALL` (nécessite permission `CALL_PHONE`).

Partager du Texte

```
1 Intent shareIntent = new Intent(Intent.ACTION_SEND);
2 shareIntent.setType("text/plain");
3 shareIntent.putExtra(Intent.EXTRA_SUBJECT, "Sujet du message");
4 shareIntent.putExtra(Intent.EXTRA_TEXT, "Contenu partager");
5 startActivity(Intent.createChooser(shareIntent, "Partager via"));
```

Passage de Données entre Activités

Envoi d'Extras

```
1 Intent intent = new Intent(this, ProfileActivity.class);
2 intent.putExtra("username", "Ali");
3 intent.putExtra("age", 25);
4 intent.putExtra("isStudent", true);
5 startActivity(intent);
```

Récupération

```

1 String user = getIntent().getStringExtra("username");
2 int age = getIntent().getIntExtra("age", 0);
3 boolean isStudent = getIntent().getBooleanExtra("isStudent", false);

```

Transmission d'Objets Complexes (Parcelable)

Parcelable est la méthode recommandée pour passer des objets.

Classe Parcelable :

```

1 public class User implements Parcelable {
2     private String name;
3     private int age;
4
5     // Constructeur, getters, setters...
6
7     protected User(Parcel in) {
8         name = in.readString();
9         age = in.readInt();
10    }
11
12    public static final Creator<User> CREATOR = new Creator<User>() {
13        @Override
14        public User createFromParcel(Parcel in) {
15            return new User(in);
16        }
17
18        @Override
19        public User[] newArray(int size) {
20            return new User[size];
21        }
22    };
23
24    @Override
25    public int describeContents() {
26        return 0;
27    }
28
29    @Override
30    public void writeToParcel(Parcel dest, int flags) {
31        dest.writeString(name);
32        dest.writeInt(age);
33    }
34 }

```

Utilisation :

```

1 // Envoi
2 User user = new User("Ali", 25);
3 intent.putExtra("user", user);
4
5 // Récupération
6 User user = getIntent().getParcelableExtra("user");

```

Activity Result API

La nouvelle méthode standard et recommandée pour gérer les résultats d'activité.

```

1  ActivityResultLauncher<Intent> launcher =
2      registerForActivityResult(
3          new ActivityResultContracts.StartActivityForResult(),
4          result -> {
5              if (result.getResultCode() == RESULT_OK) {
6                  Intent data = result.getData();
7                  if (data != null) {
8                      String resultat = data.getStringExtra("resultat");
9                      // Traitement du resultat
10                 }
11             }
12         });
13
14 // Lancer l'activité
15 Intent intent = new Intent(this, SecondActivity.class);
16 launcher.launch(intent);

```

[TIP] : Activity Result API remplace `startActivityForResult()` qui est déprécié.

Broadcast Receivers

Les Broadcasts diffusent des messages à plusieurs composants.

Types

- **Système** : Batterie, réseau, écran, démarrage, etc.
- **Personnalisé** : Créé par l'application

Exemple

Émission :

```

1  Intent intent = new Intent("com.emsi.MON_ACTION_PERSONNALISEE");
2  intent.putExtra("message", "Hello Broadcast");
3  sendBroadcast(intent);

```

Réception :

```

1  BroadcastReceiver receiver = new BroadcastReceiver() {
2      @Override
3      public void onReceive(Context context, Intent intent) {
4          String message = intent.getStringExtra("message");
5          Toast.makeText(context, message, Toast.LENGTH_SHORT).show();
6      }
7  };
8
9  IntentFilter filter = new IntentFilter("com.emsi.
10     MON_ACTION_PERSONNALISEE");
11 registerReceiver(receiver, filter);

```

[IMPORTANT] : Penser à désenregistrer le receiver dans `onDestroy()`.

PendingIntent

Un Intent différé exécuté plus tard, souvent par le système. Utilisé pour :

- Notifications
- Alarmes

— App Widgets

```
1 PendingIntent pendingIntent = PendingIntent.getActivity(
2     this, 0, intent, PendingIntent.FLAG_IMMUTABLE);
```

Chapitre 7 : API Android

Content Providers et Accès aux Contacts

Les **Content Providers** constituent le mécanisme fondamental d'Android pour partager des données entre applications.

Permissions nécessaires

```
1 <uses-permission android:name="android.permission.READ_CONTACTS" />
2 <uses-permission android:name="android.permission.WRITE_CONTACTS" />
```

Exemple : Lire les contacts

```
1 Cursor cursor = getContentResolver().query(
2     ContactsContract.Contacts.CONTENT_URI,
3     null, null, null, null
4 );
5
6 if (cursor != null && cursor.moveToFirst()) {
7     do {
8         String name = cursor.getString(
9             cursor.getColumnIndex(ContactsContract.Contacts.DISPLAY_NAME)
10        );
11        Log.d("CONTACT", name);
12    } while (cursor.moveToNext());
13    cursor.close(); // Important : lib rer les ressources
14 }
```

[IMPORTANT] : Toujours fermer le Cursor après utilisation.

Fonctions de Téléphonie

Appels téléphoniques

- **ACTION_DIAL** : Ouvre le dialer sans permission - l'utilisateur contrôle l'appel
- **ACTION_CALL** : Lance l'appel directement - nécessite CALL_PHONE permission

```
1 Intent intent = new Intent(Intent.ACTION_DIAL);
2 intent.setData(Uri.parse("tel:0612345678"));
3 startActivity(intent);
```

Envoi de SMS

```
1 SmsManager sms = SmsManager.getDefault();
2 sms.sendMessage(
3     "0612345678",
4     null,
5     "Bonjour depuis Android",
6     null, null
7 );
```

Permission nécessaire : SEND_SMS dans le manifest.

Géolocalisation avec Android

Sources de localisation

- **GPS** : Précision élevée (5-10m) mais consommation batterie importante
- **Réseau mobile** : Précision moyenne (100-500m), consommation modérée
- **Wi-Fi** : Bonne précision en zone urbaine (20-50m), faible consommation

APIs de localisation

- **LocationManager** : API historique d'Android, accès direct aux fournisseurs de localisation
- **FusedLocationProvider** : API moderne recommandée par Google. Fusionne automatiquement les sources pour optimiser précision et batterie

Permissions de localisation

- ACCESS_COARSE_LOCATION : Localisation approximative via réseau et Wi-Fi
- ACCESS_FINE_LOCATION : Localisation précise via GPS
- **Background Location** : Depuis Android 10, permission séparée pour localisation en arrière-plan

Caméra

Android permet d'accéder à l'appareil photo pour capturer des photos et vidéos. L'accès nécessite des permissions appropriées et peut se faire via :

- Intent implicite (déléguer à l'app caméra système)
- Camera2 API (contrôle avancé)
- CameraX (bibliothèque moderne recommandée)

Multimédia

Sonneries et Vibrations

```
1 // Vibration
2 Vibrator vibrator = (Vibrator) getSystemService(Context.VIBRATOR_SERVICE);
3
4 if (vibrator.hasVibrator()) {
5     vibrator.vibrate(500); // 500 millisecondes
6 }
7
8 // Sonnerie
9 MediaPlayer mediaPlayer = MediaPlayer.create(this, R.raw.son);
10 mediaPlayer.start();
```

Permission nécessaire : VIBRATE pour la vibration.

Chapitre 8 : Communication Réseau sous Android

Établissement de Connexions

Android permet d'établir des connexions à des serveurs distants via HTTP/HTTPS.

Permission nécessaire :

```
1 <uses-permission android:name="android.permission.INTERNET" />
```

Protocoles de Communication

HTTP/HTTPS

Les applications Android peuvent communiquer avec des serveurs web via les protocoles HTTP et HTTPS.

[IMPORTANT] : Utiliser HTTPS pour sécuriser les communications.

Web Services

REST et JSON

Les services web REST utilisent JSON pour l'échange de données. Android fournit des bibliothèques comme Retrofit et OkHttp pour faciliter ces interactions.

Retrofit

Retrofit est une bibliothèque moderne et puissante pour effectuer des appels réseau en Android.

Interface de service :

```
1 public interface ApiService {
2     @GET("users")
3     Call<List<User>> getUsers();
4
5     @POST("users")
6     Call<User> createUser(@Body User user);
7
8     @GET("users/{id}")
9     Call<User> getUser(@Path("id") int userId);
10 }
```

Utilisation :

```
1 Retrofit retrofit = new Retrofit.Builder()
2     .baseUrl("https://api.example.com/")
3     .addConverterFactory(GsonConverterFactory.create())
4     .build();
5
6 ApiService service = retrofit.create(ApiService.class);
7 Call<List<User>> call = service.getUsers();
8
9 call.enqueue(new Callback<List<User>>() {
10     @Override
11     public void onResponse(Call<List<User>> call, Response<List<User>>
        response) {
```

```

12         if (response.isSuccessful()) {
13             List<User> users = response.body();
14             // Traitement de la r ponse
15         }
16     }
17
18     @Override
19     public void onFailure(Call<List<User>> call, Throwable t) {
20         // Gestion des erreurs
21         Log.e("API", "Erreur: " + t.getMessage());
22     }
23 });

```

[EXPLICATION] :

- @GET, @POST : Méthodes HTTP
- @Path : Paramètre dans l'URL
- @Body : Corps de la requête
- enqueue() : Exécution asynchrone

Communication Réseau - Suite

OkHttp

OkHttp est une bibliothèque HTTP moderne pour Android, souvent utilisée avec Retrofit.

Exemple basique :

```

1 OkHttpClient client = new OkHttpClient();
2
3 Request request = new Request.Builder()
4     .url("https://api.example.com/data")
5     .build();
6
7 client.newCall(request).enqueue(new Callback() {
8     @Override
9     public void onFailure(Call call, IOException e) {
10         e.printStackTrace();
11     }
12
13     @Override
14     public void onResponse(Call call, Response response) throws
15         IOException {
16         if (response.isSuccessful()) {
17             String responseData = response.body().string();
18             // Traitement des donn es
19         }
20 });

```

Volley (Alternative)

Volley est une bibliothèque HTTP développée par Google, simple à utiliser pour des requêtes réseau.

Exemple :

```

1 RequestQueue queue = Volley.newRequestQueue(this);
2 String url = "https://api.example.com/data";

```

```

3
4 StringRequest stringRequest = new StringRequest(Request.Method.GET, url,
5     response -> {
6         // Traitement de la r ponse
7         Log.d("Volley", response);
8     },
9     error -> {
10        // Gestion des erreurs
11        Log.e("Volley", error.toString());
12    });
13
14 queue.add(stringRequest);

```

Gestion des erreurs réseau

Points importants :

- Toujours vérifier la connectivité avant les requêtes
- Gérer les timeouts
- Afficher des messages d'erreur appropriés à l'utilisateur
- Utiliser des threads secondaires pour les opérations réseau

Vérification de la connectivité :

```

1 ConnectivityManager cm = (ConnectivityManager)
2     getSystemService(Context.CONNECTIVITY_SERVICE);
3 NetworkInfo activeNetwork = cm.getActiveNetworkInfo();
4 boolean isConnected = activeNetwork != null &&
5     activeNetwork.isConnectedOrConnecting();

```

Bonnes pratiques

1. **HTTPS uniquement** : Utiliser HTTPS pour toutes les communications
2. **Threading** : Ne jamais faire de requêtes réseau sur le UI Thread
3. **Cache** : Mettre en cache les réponses pour améliorer les performances
4. **Timeouts** : Configurer des timeouts appropriés
5. **Gestion d'erreurs** : Prévoir des fallbacks et messages d'erreur clairs

Chapitre 9 : Services et Notifications

Introduction aux Services

Un **Service** est un composant Android qui exécute des opérations en arrière-plan sans interface utilisateur. Contrairement aux Activities, les Services continuent de s'exécuter même si l'utilisateur change d'application.

Types de Services

- **Started Service** : Démarré par `startService()`, continue jusqu'à ce qu'il se termine ou soit arrêté
- **Bound Service** : Lié à un composant via `bindService()`, détruit quand tous les clients se déconnectent
- **Foreground Service** : Service visible par l'utilisateur via une notification persistante

Started Service

Création d'un Service

Classe Service :

```

1 public class MonService extends Service {
2     @Override
3     public IBinder onBind(Intent intent) {
4         return null; // Service non lié
5     }
6
7     @Override
8     public int onStartCommand(Intent intent, int flags, int startId) {
9         // Code à exécuter
10        new Thread(() -> {
11            // Opération en arrière-plan
12            faireTravailLong();
13            stopSelf(); // Arrêter le service
14        }).start();
15
16        return START_STICKY; // Redémarrer si tué
17    }
18 }

```

Déclaration dans AndroidManifest.xml :

```

1 <service
2     android:name=".MonService"
3     android:enabled="true"
4     android:exported="false" />

```

Démarrer le Service :

```

1 Intent serviceIntent = new Intent(this, MonService.class);
2 startService(serviceIntent);

```

Arrêter le Service :

```

1 Intent serviceIntent = new Intent(this, MonService.class);
2 stopService(serviceIntent);

```

Flags de retour

- START_STICKY : Redémarre le service si tué, avec Intent null
- START_NOT_STICKY : Ne redémarre pas si tué
- START_REDELIVER_INTENT : Redémarre avec le même Intent

Bound Service

Un Service lié permet une communication bidirectionnelle entre le service et les composants clients.

Exemple avec Binder

Classe Service :

```

1 public class MonBoundService extends Service {
2     private final IBinder binder = new LocalBinder();
3
4     public class LocalBinder extends Binder {
5         MonBoundService getService() {
6             return MonBoundService.this;
7         }
8     }
9
10    @Override
11    public IBinder onBind(Intent intent) {
12        return binder;
13    }
14
15    public String getMessage() {
16        return "Hello from Service!";
17    }
18 }

```

Utilisation dans Activity :

```

1 private MonBoundService service;
2 private boolean bound = false;
3
4 private ServiceConnection connection = new ServiceConnection() {
5     @Override
6     public void onServiceConnected(ComponentName name, IBinder binder) {
7         MonBoundService.LocalBinder localBinder =
8             (MonBoundService.LocalBinder) binder;
9         service = localBinder.getService();
10        bound = true;
11    }
12
13    @Override
14    public void onServiceDisconnected(ComponentName name) {
15        bound = false;
16    }
17 };
18
19 @Override
20 protected void onStart() {
21     super.onStart();
22     Intent intent = new Intent(this, MonBoundService.class);
23     bindService(intent, connection, Context.BIND_AUTO_CREATE);
24 }
25
26 @Override
27 protected void onStop() {
28     super.onStop();
29     if (bound) {
30         unbindService(connection);
31         bound = false;
32     }
33 }

```

Foreground Service

Les Foreground Services sont visibles par l'utilisateur via une notification persistante. Depuis Android 8.0 (API 26), ils sont obligatoires pour les services en arrière-plan.

Création

```

1 public class MonForegroundService extends Service {
2     @Override
3     public int onStartCommand(Intent intent, int flags, int startId) {
4         // Cr e r la notification
5         Notification notification = new NotificationCompat.Builder(this,
6             "channel_id")
7             .setContentTitle("Service en cours")
8             .setContentText("Travail en arri re -plan")
9             .setSmallIcon(R.drawable.ic_notification)
10            .build();
11
12        // D marrer en foreground
13        startForeground(1, notification);
14
15        // Faire le travail
16        faireTravail();
17
18        return START_STICKY;
19    }
20
21    @Override
22    public IBinder onBind(Intent intent) {
23        return null;
24    }
25 }

```

[IMPORTANT] : Depuis Android 8.0, créer un canal de notification avant d'afficher la notification.

Notifications

Les notifications permettent d'informer l'utilisateur d'événements même quand l'application n'est pas au premier plan.

Création d'un canal de notification (Android 8.0+)

```

1 private void createNotificationChannel() {
2     if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
3         CharSequence name = "Mon Canal";
4         String description = "Description du canal";
5         int importance = NotificationManager.IMPORTANCE_DEFAULT;
6         NotificationChannel channel = new NotificationChannel(
7             "channel_id", name, importance);
8         channel.setDescription(description);
9
10        NotificationManager notificationManager =
11            getSystemService(NotificationManager.class);
12        notificationManager.createNotificationChannel(channel);
13    }
14 }

```


Création d'une notification simple

```
1 NotificationCompat.Builder builder =
2     new NotificationCompat.Builder(this, "channel_id")
3     .setSmallIcon(R.drawable.ic_notification)
4     .setContentTitle("Titre de la notification")
5     .setContentText("Contenu de la notification")
6     .setPriority(NotificationCompat.PRIORITY_DEFAULT);
7
8 NotificationManagerCompat notificationManager =
9     NotificationManagerCompat.from(this);
10 notificationManager.notify(1, builder.build());
```

Notification avec action

```
1 Intent intent = new Intent(this, MainActivity.class);
2 PendingIntent pendingIntent = PendingIntent.getActivity(
3     this, 0, intent, PendingIntent.FLAG_IMMUTABLE);
4
5 Intent actionIntent = new Intent(this, ActionReceiver.class);
6 PendingIntent actionPendingIntent = PendingIntent.getBroadcast(
7     this, 0, actionIntent, PendingIntent.FLAG_IMMUTABLE);
8
9 Notification notification = new NotificationCompat.Builder(this,
10     "channel_id")
11     .setSmallIcon(R.drawable.ic_notification)
12     .setContentTitle("Notification avec action")
13     .setContentText("Cliquez pour ouvrir")
14     .setContentIntent(pendingIntent)
15     .addAction(R.drawable.ic_action, "Action", actionPendingIntent)
16     .setAutoCancel(true)
17     .build();
```

Types de notifications

- **Notification simple** : Message basique
- **Notification avec action** : Boutons d'action intégrés
- **Notification BigText** : Texte long dépliant
- **Notification BigPicture** : Image grande taille
- **Notification avec progression** : Barre de progression

Notification BigText

```
1 NotificationCompat.BigTextStyle bigTextStyle =
2     new NotificationCompat.BigTextStyle();
3 bigTextStyle.bigText("Texte très long qui sera affiché " +
4     "en entier quand l'utilisateur déplie la notification...");
5
6 Notification notification = new NotificationCompat.Builder(this,
7     "channel_id")
8     .setSmallIcon(R.drawable.ic_notification)
9     .setContentTitle("Notification BigText")
10     .setStyle(bigTextStyle)
11     .build();
```

Notification avec progression

```

1 NotificationCompat.Builder builder =
2     new NotificationCompat.Builder(this, "channel_id")
3     .setSmallIcon(R.drawable.ic_notification)
4     .setContentTitle("T l chargement")
5     .setContentText("En cours...")
6     .setProgress(100, 50, false); // max, current, indeterminate
7
8 // Mettre jour la progression
9 builder.setProgress(100, 75, false);
10 notificationManager.notify(1, builder.build());

```

JobScheduler et WorkManager

WorkManager (Recommandé)

WorkManager est la solution moderne recommandée pour les tâches en arrière-plan.

Exemple :

```

1 OneTimeWorkRequest uploadWork = new OneTimeWorkRequest.Builder(
2     UploadWorker.class)
3     .setConstraints(new Constraints.Builder()
4         .setRequiredNetworkType(NetworkType.CONNECTED)
5         .build())
6     .build();
7
8 WorkManager.getInstance(this).enqueue(uploadWork);

```

Classe Worker :

```

1 public class UploadWorker extends Worker {
2     public UploadWorker(@NonNull Context context,
3         @NonNull WorkerParameters params) {
4         super(context, params);
5     }
6
7     @NonNull
8     @Override
9     public Result doWork() {
10         // T che en arri re-plan
11         try {
12             uploadData();
13             return Result.success();
14         } catch (Exception e) {
15             return Result.retry();
16         }
17     }
18 }

```

Bonnes pratiques

1. Utiliser WorkManager pour les tâches en arrière-plan
2. Créer des canaux de notification pour Android 8.0+
3. Arrêter proprement les services dans onDestroy()
4. Libérer les ressources (unbind, stopSelf)

5. Gérer les permissions nécessaires
6. Tester sur différentes versions d'Android

Permissions pour les Services

```
1 <!-- Pour les services foreground -->
2 <uses-permission android:name="android.permission.FOREGROUND_SERVICE" />
3
4 <!-- Pour les notifications -->
5 <uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

[IMPORTANT] : Depuis Android 13, la permission POST_NOTIFICATIONS doit être demandée à l'exécution.

Conclusion

Ce résumé couvre les concepts fondamentaux du développement Android du chapitre 1 au chapitre 9. Les points clés à retenir pour l'examen sont :

- **Architecture Android** : Composants fondamentaux (Activity, Service, BroadcastReceiver, ContentProvider)
- **Cycle de vie** : Méthodes du cycle de vie des Activities
- **Interface utilisateur** : Layouts, RecyclerView, Adapters, événements
- **Persistance** : SharedPreferences, SQLite, fichiers
- **Threading** : UI Thread, Handler, ExecutorService
- **Intents** : Navigation, communication entre composants
- **API Android** : Contacts, téléphonie, localisation, caméra
- **Réseau** : HTTP/HTTPS, Retrofit, OkHttp
- **Services** : Started, Bound, Foreground Services, Notifications

[CONSEILS POUR L'EXAMEN] :

1. Maîtriser le cycle de vie des Activities
2. Comprendre les différents types de Layouts et leurs usages
3. Savoir utiliser RecyclerView et Adapters
4. Connaître les méthodes de persistance des données
5. Comprendre la gestion des threads et la communication réseau
6. Maîtriser les Intents (explicites et implicites)
7. Connaître les Services et Notifications

Bon courage pour vos révisions !